Definitions in `Sarason/` and `R2.lean`

Lean Formalization Project

October 20, 2026

This document provides a comprehensive list of all definitions manually formalized in the `ComplexAnalysis` project, specifically within `R2.lean` and the `Sarason/` directory. For each definition, the raw Lean code is presented first, followed by its mathematical interpretation.

1 `R2.lean`

- `sqNorm` ($x \in \mathbb{R}^2 \to \mathbb{R}$): The squared Euclidean norm.

```
noncomputable def sqNorm (x : ℝ × ℝ) : ℝ := x.1^2 + x.2^2
```

$$\text{sqNorm}(x) = x_1^2 + x_2^2$$

- `sqDist` ($x, y \in \mathbb{R}^2 \to \mathbb{R}$): The squared Euclidean distance.

```
noncomputable def sqDist (x y : ℝ × ℝ) : ℝ :=
  (x.1 - y.1)^2 + (x.2 - y.2)^2
```

$$\text{sqDist}(x, y) = \text{sqNorm}(x - y)$$

- `euclideanNorm` ($x \in \mathbb{R}^2 \to \mathbb{R}$): The standard Euclidean norm.

```
noncomputable def euclideanNorm (x : ℝ × ℝ) : ℝ :=
  Real.sqrt (sqNorm x)
```

$$\text{euclideanNorm}(x) = \sqrt{x_1^2 + x_2^2}$$

- `euclideanDist` ($x, y \in \mathbb{R}^2 \to \mathbb{R}$): The Euclidean distance.

```
noncomputable def euclideanDist (x y : ℝ × ℝ) : ℝ :=
  Real.sqrt (sqDist x y)
```

$$\text{euclideanDist}(x, y) = \sqrt{\text{sqDist}(x, y)}$$

- `LimitRtoR2` ($f : \mathbb{R} \to \mathbb{R}^2$, $a \in \mathbb{R}$, $L \in \mathbb{R}^2$):

```
def LimitRtoR2 (f : ℝ → ℝ × ℝ) (a : ℝ) (L : ℝ × ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ, 0 < |x - a| ∧ |x - a| < δ → euclideanDist (f x) L
    < ε
```

$$\lim_{x \to a} f(x) = L \iff \forall \epsilon > 0, \exists \delta > 0, \forall x, 0 < |x - a| < \delta \implies \text{euclideanDist}(f(x), L) < \epsilon$$

- `LimitR2toR` ($f : \mathbb{R}^2 \to \mathbb{R}$, $a \in \mathbb{R}^2$, $L \in \mathbb{R}$):

```
def LimitR2toR (f : ℝ × ℝ → ℝ) (a : ℝ × ℝ) (L : ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ × ℝ, 0 < euclideanDist x a ∧ euclideanDist x a < δ
    → |f x - L| < ε
```

$$\lim_{x\to a} f(x) = L \iff \forall \epsilon > 0, \exists \delta > 0, \forall x \in \mathbb{R}^2, 0 < \text{euclideanDist}(x, a) < \delta \implies |f(x) - L| < \epsilon$$

- `LimitR2toR2` ($f : \mathbb{R}^2 \to \mathbb{R}^2$, $a \in \mathbb{R}^2$, $L \in \mathbb{R}^2$):

```
def LimitR2toR2 (f : ℝ × ℝ → ℝ × ℝ) (a : ℝ × ℝ) (L : ℝ × ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ × ℝ, 0 < euclideanDist x a ∧ euclideanDist x a < δ
    → euclideanDist (f x) L < ε
```

$$\lim_{x\to a} f(x) = L \iff \forall \epsilon > 0, \exists \delta > 0, \forall x \in \mathbb{R}^2, 0 < \text{euclideanDist}(x, a) < \delta \implies \text{euclideanDist}(f(x), L) < \epsilon$$

- `ConvergesR2` ($\text{seq} : \mathbb{N} \to \mathbb{R}^2$, $L \in \mathbb{R}^2$):

```
def ConvergesR2 (seq : ℕ → ℝ × ℝ) (L : ℝ × ℝ): Prop :=
  ∀ ε > 0, ∃ N : ℕ, ∀ n ≥ N, euclideanDist (seq n) L < ε
```

$$\lim_{n\to\infty} \text{seq}(n) = L \iff \forall \epsilon > 0, \exists N \in \mathbb{N}, \forall n \geq N, \text{euclideanDist}(\text{seq}(n), L) < \epsilon$$

- `HasFDerivAt_R2_eps` ($u, v : \mathbb{R}^2 \to \mathbb{R}$, $u_{x0}, u_{y0}, v_{x0}, v_{y0} \in \mathbb{R}$, $a \in \mathbb{R}^2$): Defines the Fréchet derivative of $F(x, y) = (u(x, y), v(x, y))$ at a. Let $L(h) = \begin{pmatrix} u_{x0} & u_{y0} \\ v_{x0} & v_{y0} \end{pmatrix} h$.

```
def HasFDerivAt_R2_eps (u v : ℝ × ℝ → ℝ) (ux₀ uy₀ vx₀ vy₀ : ℝ) (a : ℝ × ℝ) :
    Prop :=
  LimitR2toR (fun h =>
    euclideanNorm (
      (u (a.1 + h.1, a.2 + h.2) - u a) - (ux₀ * h.1 + uy₀ * h.2),
      (v (a.1 + h.1, a.2 + h.2) - v a) - (vx₀ * h.1 + vy₀ * h.2)
    ) / euclideanNorm h
  ) (0, 0) 0
```

$$\lim_{h\to 0} \frac{\left\| F(a+h) - F(a) - \begin{pmatrix} u_{x0} & u_{y0} \\ v_{x0} & v_{y0} \end{pmatrix} h \right\|}{\|h\|} = 0$$

$$\lim_{(h_1,h_2)\to(0,0)} \frac{\left\| \begin{pmatrix} u(a_1+h_1, a_2+h_2) - u(a_1, a_2) \\ v(a_1+h_1, a_2+h_2) - v(a_1, a_2) \end{pmatrix} - \begin{pmatrix} u_{x0} & u_{y0} \\ v_{x0} & v_{y0} \end{pmatrix} \begin{pmatrix} h_1 \\ h_2 \end{pmatrix} \right\|}{\|(h_1, h_2)\|} = 0$$

- `DifferentiableAt_R2_eps` ($u, v : \mathbb{R}^2 \to \mathbb{R}$, $a \in \mathbb{R}^2$):

```
def DifferentiableAt_R2_eps (u v : ℝ × ℝ → ℝ) (a : ℝ × ℝ) : Prop :=
  ∃ (ux₀ uy₀ vx₀ vy₀ : ℝ), HasFDerivAt_R2_eps u v ux₀ uy₀ vx₀ vy₀ a
```

$$\exists u_{x0}, u_{y0}, v_{x0}, v_{y0} \in \mathbb{R}, \text{HasFDerivAt_R2_eps}(u, v, u_{x0}, u_{y0}, v_{x0}, v_{y0}, a)$$

- `HasDerivAt_RtoR2_eps` ($f : \mathbb{R} \to \mathbb{R}^2$, $f_0' \in \mathbb{R}^2$, $t_0 \in \mathbb{R}$): Defines the derivative of a parametric curve in $\mathbb{R}^2$.

```
def HasDerivAt_RtoR2_eps (f : ℝ → ℝ × ℝ) (f'₀ : ℝ × ℝ) (t₀ : ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ t : ℝ, 0 < |t - t₀| ∧ |t - t₀| < δ →
    euclideanDist (f t) (f t₀ + (f'₀.1 * (t - t₀), f'₀.2 * (t - t₀))) < ε * |t
    - t₀|
```

$$\lim_{t \to t_0} (f(t) - f(t_0) - (t - t_0) f'_0) = 0$$

- `HasPartialDerivX_R2_eps` ($u : \mathbb{R}^2 \to \mathbb{R}$, $ux_0 \in \mathbb{R}$, $p_0 \in \mathbb{R}^2$):

```
def HasPartialDerivX_R2_eps (u : ℝ × ℝ → ℝ) (ux₀ : ℝ) (p₀ : ℝ × ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ, 0 < |x - p₀.1| ∧ |x - p₀.1| < δ →
    |(u (x, p₀.2) - u p₀) / (x - p₀.1) - ux₀| < ε
```

$$\lim_{x \to p_{0,x}} \frac{u(x, p_{0,y}) - u(p_0)}{x - p_{0,x}} = ux_0$$

- `HasPartialDerivY_R2_eps` ($u : \mathbb{R}^2 \to \mathbb{R}$, $uy_0 \in \mathbb{R}$, $p_0 \in \mathbb{R}^2$):

```
def HasPartialDerivY_R2_eps (u : ℝ × ℝ → ℝ) (uy₀ : ℝ) (p₀ : ℝ × ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ y : ℝ, 0 < |y - p₀.2| ∧ |y - p₀.2| < δ →
    |(u (p₀.1, y) - u p₀) / (y - p₀.2) - uy₀| < ε

/-- A real-valued function of two real variables is harmonic on an open set G
if its first and second partial derivatives are continuous and satisfy Laplace
   's equation. -/
```

$$\lim_{y \to p_{0,y}} \frac{u(p_{0,x}, y) - u(p_0)}{y - p_{0,y}} = uy_0$$

- `TwiceContinuouslyDifferentiable_R2_eps` ($u : \mathbb{R}^2 \to \mathbb{R}$, $G \subset \mathbb{R}^2$): Defines twice continuous differentiability (C^2) for a real-valued function on an open subset $G \subset \mathbb{R}^2$.

```
def TwiceContinuouslyDifferentiable_R2_eps (u : ℝ × ℝ → ℝ) (G : Set (ℝ × ℝ))
    : Prop :=
  IsOpen G ∧
  ∃ (ux uy uxx uxy uyx uyy : ℝ × ℝ → ℝ),
    (∀ p ∈ G, HasPartialDerivX_R2_eps u (ux p) p) ∧
    (∀ p ∈ G, HasPartialDerivY_R2_eps u (uy p) p) ∧
    (ContinuousOn ux G) ∧
    (ContinuousOn uy G) ∧
    (∀ p ∈ G, HasPartialDerivX_R2_eps ux (uxx p) p) ∧
    (∀ p ∈ G, HasPartialDerivY_R2_eps ux (uxy p) p) ∧
    (∀ p ∈ G, HasPartialDerivX_R2_eps uy (uyx p) p) ∧
    (∀ p ∈ G, HasPartialDerivY_R2_eps uy (uyy p) p) ∧
    (ContinuousOn uxx G) ∧
    (ContinuousOn uxy G) ∧
    (ContinuousOn uyx G) ∧
    (ContinuousOn uyy G)
```

$u \in C^2(G, \mathbb{R}) \iff u_x, u_y, u_{xx}, u_{xy}, u_{yx}, u_{yy}$ exist and are continuous on the open set G

- `Harmonic_R2_eps` ($u : \mathbb{R}^2 \to \mathbb{R}$, $G \subset \mathbb{R}^2$):

```
def Harmonic_R2_eps (u : ℝ × ℝ → ℝ) (G : Set (ℝ × ℝ)) : Prop :=
  IsOpen G ∧
  ∃ (ux uy uxx uxy uyx uyy : ℝ × ℝ → ℝ),
    (∀ p ∈ G, HasPartialDerivX_R2_eps u (ux p) p) ∧
    (∀ p ∈ G, HasPartialDerivY_R2_eps u (uy p) p) ∧
    (ContinuousOn ux G) ∧
    (ContinuousOn uy G) ∧
    (∀ p ∈ G, HasPartialDerivX_R2_eps ux (uxx p) p) ∧
    (∀ p ∈ G, HasPartialDerivY_R2_eps ux (uxy p) p) ∧
    (∀ p ∈ G, HasPartialDerivX_R2_eps uy (uyx p) p) ∧
```

```
    (∀ p ∈ G, HasPartialDerivY_R2_eps uy (uyy p) p) ∧
    (ContinuousOn uxx G) ∧
    (ContinuousOn uxy G) ∧
    (ContinuousOn uyx G) ∧
    (ContinuousOn uyy G) ∧
    (∀ p ∈ G, uxx p + uyy p = 0)
```

First and second partials continuous on G, $\quad \Delta u = u_{xx} + u_{yy} = 0$

2 Sarason/Definitions.lean

- `HasLimitAt_eps` ($f : \mathbb{C} \to \mathbb{C}$, $L \in \mathbb{C}$, $z_0 \in \mathbb{C}$):

```
def HasLimitAt_eps (f : ℂ → ℂ) (L : ℂ) (z₀ : ℂ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ z, 0 < ‖z - z₀‖ ∧ ‖z - z₀‖ < δ → ‖f z - L‖ < ε

--  Weierstrass definition of a limit at infinity.
```

$$\lim_{z \to z_0} f(z) = L$$

- `HasLimitAtInf_eps` ($f : \mathbb{C} \to \mathbb{R}$, $L \in \mathbb{R}$):

```
def HasLimitAtInf_eps (f : ℂ → ℝ) (L : ℝ) : Prop :=
  ∀ ε > 0, ∃ M > 0, ∀ z, ‖z‖ > M → |f z - L| < ε

--  Weierstrass definition of having a complex derivative at z₀.
```

$$\lim_{|z| \to \infty} f(z) = L$$

- `HasDerivAt_eps` ($f : \mathbb{C} \to \mathbb{C}$, $f_0' \in \mathbb{C}$, $z_0 \in \mathbb{C}$): Complex differentiability at a point.

```
def HasDerivAt_eps (f : ℂ → ℂ) (f'₀ : ℂ) (z₀ : ℂ) : Prop :=
  HasLimitAt_eps (fun z => (f z - f z₀) / (z - z₀)) f'₀ z₀

--  Weierstrass definition of complex differentiability at z₀.
```

$$\lim_{z \to z_0} \frac{f(z) - f(z_0)}{z - z_0} = f_0'$$

- `DifferentiableAt_eps` ($f : \mathbb{C} \to \mathbb{C}$, $z_0 \in \mathbb{C}$):

```
def DifferentiableAt_eps (f : ℂ → ℂ) (z₀ : ℂ) : Prop :=
  ∃ f'₀, HasDerivAt_eps f f'₀ z₀

-- Weierstrass definition of partial derivative with respect to x.
```

f is differentiable at z_0

- `HasPartialDerivX_C_to_R_eps` / `HasPartialDerivY_C_to_R_eps`: Partial derivatives of a real-valued function of a complex variable (identifying $\mathbb{C} \cong \mathbb{R}^2$).

```
def HasPartialDerivX_C_to_R_eps (u : ℂ → ℝ) (ux₀ : ℝ) (z₀ : ℂ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ, 0 < |x - z₀.re| ∧ |x - z₀.re| < δ →
    |(u ((x : ℂ) + (z₀.im : ℂ) * I) - u z₀) / (x - z₀.re) - ux₀| < ε

-- Weierstrass definition of partial derivative with respect to y.
```

$$\frac{\partial u}{\partial x}(z_0) = u_{x0}$$

- `HasPartialDerivY_C_to_R_eps`

```
def HasPartialDerivY_C_to_R_eps (u : ℂ → ℝ) (uy₀ : ℝ) (z₀ : ℂ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ y : ℝ, 0 < |y - z₀.im| ∧ |y - z₀.im| < δ →
    |(u ((z₀.re : ℂ) + (y : ℂ) * I) - u z₀) / (y - z₀.im) - uy₀| < ε

--  Equivalence theorem between Mathlib's limit at a point (using punctured
    neighborhoods) and Weierstrass epsilon-delta limit.
```

$$\frac{\partial u}{\partial y}(z_0) = u_{y0}$$

- `deriv_eps` ($f : \mathbb{C} \to \mathbb{C}$, $z_0 \in \mathbb{C}$, h : `DifferentiableAt_eps` f z_0): The classical derivative value $f'(z_0)$ extracted via choice.

```
noncomputable def deriv_eps (f : ℂ → ℂ) (z₀ : ℂ) (h : DifferentiableAt_eps f
    z₀) : ℂ :=
  Classical.choose h
```

$$\text{deriv_eps}(f, z_0) = f'(z_0)$$

- `HasDerivAt_R_to_C_eps` ($f : \mathbb{R} \to \mathbb{C}$, $f'_0 \in \mathbb{C}$, $t_0 \in \mathbb{R}$): Derivative of a complex-valued path parameterized by a real variable.

```
def HasDerivAt_R_to_C_eps (f : ℝ → ℂ) (f'₀ : ℂ) (t₀ : ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ t : ℝ, 0 < |t - t₀| ∧ |t - t₀| < δ → ‖(f t - f t₀) / ((t
    - t₀ : ℝ) : ℂ) - f'₀‖ < ε
```

$$\lim_{t \to t_0} \frac{f(t) - f(t_0)}{t - t_0} = f'_0$$

- `DifferentiableAt_R_to_C_eps` ($f : \mathbb{R} \to \mathbb{C}$, $t_0 \in \mathbb{R}$):

```
def DifferentiableAt_R_to_C_eps (f : ℝ → ℂ) (t₀ : ℝ) : Prop :=
  ∃ f'₀, HasDerivAt_R_to_C_eps f f'₀ t₀

/-- The classical derivative value extracted via choice, or 0 if not
    differentiable. -/
```

$$\exists f'_0 \in \mathbb{C}, \text{HasDerivAt_R_to_C_eps}(f, f'_0, t_0)$$

- `deriv_R_to_C_eps` ($f : \mathbb{R} \to \mathbb{C}$, $t_0 \in \mathbb{R}$):

```
noncomputable def deriv_R_to_C_eps (f : ℝ → ℂ) (t₀ : ℝ) : ℂ :=
  if h : DifferentiableAt_R_to_C_eps f t₀ then Classical.choose h else 0
```

$$\text{deriv_R_to_C_eps}(f, t_0) = \begin{cases} f'(t_0) & \text{if differentiable} \\ 0 & \text{otherwise} \end{cases}$$

- `HolomorphicAt_eps` ($f : \mathbb{C} \to \mathbb{C}$, $z_0 \in \mathbb{C}$): f is complex-differentiable in some open neighborhood around z_0.

```
def HolomorphicAt_eps (f : ℂ → ℂ) (z₀ : ℂ) : Prop :=
  DifferentiableAt_eps f z₀

/-- A function is holomorphic on a set if it is complex differentiable at
    every point of the set. -/
```

f is holomorphic at $z_0 \iff f$ is complex-differentiable on some open ball around z_0

- `HolomorphicOn_eps` ($f : \mathbb{C} \to \mathbb{C}$, $G \subset \mathbb{C}$): f is holomorphic at every point in the open set G.

```
def HolomorphicOn_eps (f : ℂ → ℂ) (G : Set ℂ) : Prop :=
  ∀ z ∈ G, DifferentiableAt_eps f z
```

f is holomorphic on G

- `polar_partial_r` ($ux, uy : \mathbb{R}^2 \to \mathbb{R}$, $p \in \mathbb{R}^2$): Directional derivative in the radial direction for polar coordinates.

```
noncomputable def polar_partial_r (ux uy : ℝ × ℝ → ℝ) (p : ℝ × ℝ) : ℝ :=
  ux (p.1 * Real.cos p.2, p.1 * Real.sin p.2) * Real.cos p.2 +
  uy (p.1 * Real.cos p.2, p.1 * Real.sin p.2) * Real.sin p.2
```

$$U_r(r, \theta) = u_x(r\cos\theta, r\sin\theta)\cos\theta + u_y(r\cos\theta, r\sin\theta)\sin\theta$$

- `polar_partial_theta` ($ux, uy : \mathbb{R}^2 \to \mathbb{R}$, $p \in \mathbb{R}^2$): Directional derivative in the angular direction for polar coordinates.

```
noncomputable def polar_partial_theta (ux uy : ℝ × ℝ → ℝ) (p : ℝ × ℝ) : ℝ
    :=
  -ux (p.1 * Real.cos p.2, p.1 * Real.sin p.2) * p.1 * Real.sin p.2 +
  uy (p.1 * Real.cos p.2, p.1 * Real.sin p.2) * p.1 * Real.cos p.2
```

$$U_\theta(r, \theta) = -r\sin\theta\ u_x(r\cos\theta, r\sin\theta) + r\cos\theta\ u_y(r\cos\theta, r\sin\theta)$$

- `polar_partial_rr` ($uxx, uxy, uyy : \mathbb{R}^2 \to \mathbb{R}$, $p \in \mathbb{R}^2$): Second partial derivative with respect to r in polar coordinates.

```
noncomputable def polar_partial_rr (uxx uxy uyy : ℝ × ℝ → ℝ) (p : ℝ × ℝ) :
    ℝ :=
  Real.cos p.2 * polar_partial_r uxx uxy p + Real.sin p.2 * polar_partial_r
    uxy uyy p
```

$$U_{rr}(r, \theta) = \cos\theta\,(u_{xx})_r + \sin\theta\,(u_{xy})_r = \cos^2\theta\,u_{xx} + 2\sin\theta\cos\theta\,u_{xy} + \sin^2\theta\,u_{yy}$$

- `polar_partial_thetatheta` ($ux, uy, uxx, uxy, uyy : \mathbb{R}^2 \to \mathbb{R}$, $p \in \mathbb{R}^2$): Second partial derivative with respect to θ in polar coordinates.

```
noncomputable def polar_partial_thetatheta (ux uy uxx uxy uyy : ℝ × ℝ → ℝ) (
    p : ℝ × ℝ) : ℝ :=
  -p.1 * Real.cos p.2 * ux (p.1 * Real.cos p.2, p.1 * Real.sin p.2)
  - p.1 * Real.sin p.2 * polar_partial_theta uxx uxy p
  - p.1 * Real.sin p.2 * uy (p.1 * Real.cos p.2, p.1 * Real.sin p.2)
  + p.1 * Real.cos p.2 * polar_partial_theta uxy uyy p
```

$$U_{\theta\theta}(r, \theta) = -r(u_x\cos\theta + u_y\sin\theta) - r\sin\theta\,(u_{xx})_\theta + r\cos\theta\,(u_{xy})_\theta$$

- `TwiceContinuouslyDifferentiable_C_eps` ($f : \mathbb{C} \to \mathbb{C}$, $G \subset \mathbb{C}$): f is twice continuously differentiable on the open set G if its real and imaginary parts are.

```
def TwiceContinuouslyDifferentiable_C_eps (f : ℂ → ℂ) (G : Set ℂ) : Prop :=
  ComplexAnalysis.R2.TwiceContinuouslyDifferentiable_R2_eps (fun p => (f (p.1
    + p.2 * I)).re) {p : ℝ × ℝ | p.1 + p.2 * I ∈ G} ∧
  ComplexAnalysis.R2.TwiceContinuouslyDifferentiable_R2_eps (fun p => (f (p.1
    + p.2 * I)).im) {p : ℝ × ℝ | p.1 + p.2 * I ∈ G}
```

$$\mathrm{Re}(f), \mathrm{Im}(f) \in C^2(G, \mathbb{R})$$

- `Harmonic_C_eps` ($f : \mathbb{C} \to \mathbb{C}$, $G \subset \mathbb{C}$): f is harmonic on the open set G if its real and imaginary parts satisfy Laplace's equation $\Delta = 0$ on G.

```
def Harmonic_C_eps (f : ℂ → ℂ) (G : Set ℂ) : Prop :=
  ComplexAnalysis.R2.Harmonic_R2_eps (fun p => (f (p.1 + p.2 * I)).re) {p : ℝ
    × ℝ | p.1 + p.2 * I ∈ G} ∧
  ComplexAnalysis.R2.Harmonic_R2_eps (fun p => (f (p.1 + p.2 * I)).im) {p : ℝ
    × ℝ | p.1 + p.2 * I ∈ G}
```

$$\Delta \mathrm{Re}(f) = 0, \quad \Delta \mathrm{Im}(f) = 0 \text{ on } G$$

- `HarmonicConjugate_C_eps` ($u, v : \mathbb{C} \to \mathbb{R}$, $G \subset \mathbb{C}$): A real-valued function v is a harmonic conjugate of a real-valued function u on an open set G if both are harmonic on G and they satisfy the Cauchy-Riemann equations on G.

```
def HarmonicConjugate_C_eps (u v : ℂ → ℝ) (G : Set ℂ) : Prop :=
  ComplexAnalysis.R2.Harmonic_R2_eps (fun p => u (p.1 + p.2 * I)) {p : ℝ × ℝ
    | p.1 + p.2 * I ∈ G} ∧
  ComplexAnalysis.R2.Harmonic_R2_eps (fun p => v (p.1 + p.2 * I)) {p : ℝ × ℝ
    | p.1 + p.2 * I ∈ G} ∧
  ∃ (ux uy vx vy : ℂ → ℝ),
    (∀ z ∈ G, HasPartialDerivX_C_to_R_eps u (ux z) z) ∧
    (∀ z ∈ G, HasPartialDerivY_C_to_R_eps u (uy z) z) ∧
    (∀ z ∈ G, HasPartialDerivX_C_to_R_eps v (vx z) z) ∧
    (∀ z ∈ G, HasPartialDerivY_C_to_R_eps v (vy z) z) ∧
    (∀ z ∈ G, ux z = vy z ∧ uy z = -vx z)
```

$$\Delta u = 0, \quad \Delta v = 0, \quad u_x = v_y, \quad u_y = -v_x \text{ on } G$$

3 Sarason/Chapter1.lean

- `chordalMetric` ($z, w \in \mathbb{C} \to \mathbb{R}$): The chordal distance between two points in the complex plane mapped to the Riemann sphere.

```
noncomputable def chordalMetric (z w : ℂ) : ℝ :=
  (2 * Complex.abs (z - w)) / (Real.sqrt (1 + Complex.normSq z) * Real.sqrt (1
    + Complex.normSq w))
```

$$\chi(z, w) = \frac{2|z - w|}{\sqrt{1 + |z|^2}\sqrt{1 + |w|^2}}$$

- `chordalMetricToInf` ($z \in \mathbb{C} \to \mathbb{R}$): The chordal distance from a point z to the point at infinity ∞.

```
noncomputable def chordalMetricToInf (z : ℂ) : ℝ :=
  2 / Real.sqrt (1 + Complex.normSq z)
```

$$\chi(z, \infty) = \frac{2}{\sqrt{1 + |z|^2}}$$

- `Φ (Stereographic Projection)`

```
noncomputable def Φ (z : ℂ) : EuclideanSpace ℝ (Fin 3) :=
  ![ (2 * z.re) / (1 + Complex.normSq z),
     (2 * z.im) / (1 + Complex.normSq z),
     (Complex.normSq z - 1) / (1 + Complex.normSq z) ]
```

$$\Phi(z) = \left(\frac{2\,\mathrm{Re}(z)}{1+|z|^2}, \frac{2\,\mathrm{Im}(z)}{1+|z|^2}, \frac{|z|^2-1}{1+|z|^2} \right) \in S^2$$

- `ChordalC`: Type alias for $\mathbb{C}$ equipped with the chordal metric.

```
def Chordalℂ : Type := ℂ
```

$$\text{ChordalC} = \mathbb{C} \text{ (equipped with chordal metric)}$$

4 Sarason/Chapter2.lean

- `del` (∂)

```
def del (f : ℂ → ℂ) (z : ℂ) : ℂ :=
  let L := fderiv ℝ f z
  let df_dx := L 1
  let df_dy := L I
  (1 / 2 : ℂ) * (df_dx - I * df_dy)

/-- The ∂ operator (del-bar) for a function f : ℂ → ℂ at point z. -/
```

$$\partial f = \frac{1}{2}\left(\frac{\partial f}{\partial x} - i \frac{\partial f}{\partial y} \right)$$

- `delBar` ($\bar{\partial}$)

```
def delBar (f : ℂ → ℂ) (z : ℂ) : ℂ :=
  let L := fderiv ℝ f z
  let df_dx := L 1
  let df_dy := L I
  (1 / 2 : ℂ) * (df_dx + I * df_dy)

-- Local notation for Sarason's flavor
local notation f "z" => del f
local notation f "zbar" => delBar f

/--
  Relating Sarason's Weierstrass derivative to Mathlib's Filter-based
    HasDerivAt.
-/
```

$$\bar{\partial} f = \frac{1}{2}\left(\frac{\partial f}{\partial x} + i \frac{\partial f}{\partial y} \right)$$

- `ConformalLinearMap_eps` ($ux, uy, vx, vy \in \mathbb{R}$, $v \in \mathbb{C}$): Real-linear transformation on $\mathbb{C} \cong \mathbb{R}^2$ specified by the 2×2 Jacobian entries (u_x, u_y, v_x, v_y).

```
def ConformalLinearMap_eps (ux uy vx vy : ℝ) (v : ℂ) : ℂ :=
  (ux * v.re + uy * v.im) + I * (vx * v.re + vy * v.im)
```

$$L(v) = (u_x\,\mathrm{Re}(v) + u_y\,\mathrm{Im}(v)) + i(v_x\,\mathrm{Re}(v) + v_y\,\mathrm{Im}(v)) = (\partial f)\,v + (\bar{\partial} f)\,\bar{v}$$

- `del_eps` / `delBar_eps` ($u_{x0}, u_{y0}, v_{x0}, v_{y0} \in \mathbb{R}$): Wirtinger derivatives expressed purely in terms of the real partial derivatives of $f = u + iv$.

```
noncomputable def del_eps (ux uy vx vy : ℝ) : ℂ :=
  (1 / 2 : ℂ) * ( (ux + vy) + I * (vx - uy) )
```

$$\partial f = \frac{1}{2}(u_{x0} + v_{y0}) + \frac{i}{2}(v_{x0} - u_{y0})$$

- `delBar_eps`

```
noncomputable def delBar_eps (ux uy vx vy : ℝ) : ℂ :=
  (1 / 2 : ℂ) * ( (ux - vy) + I * (vx + uy) )
```

$$\bar{\partial} f = \frac{1}{2}(u_{x0} - v_{y0}) + \frac{i}{2}(v_{x0} + u_{y0})$$

- `path_in_C` ($x, y \in \mathbb{C}$): A continuous path $\gamma : [0,1] \to \mathbb{C}$ starting at x and ending at y.

```
def path_in_C (x y : ℂ) : Type := Path x y

/-- Velocity / derivative of a path γ : path_in_C x y at parameter time t ∈ ℝ
    . -/
```

$$\gamma : [0,1] \to \mathbb{C} \text{ where } \gamma(0) = x, \gamma(1) = y$$

- `translated_path` ($c, x, y \in \mathbb{C}$, γ : path_in_C x y): A path shifted by the constant c, creating a path from $c + x$ to $c + y$.

```
noncomputable def translated_path {x y : ℂ} (c : ℂ) (γ : path_in_C x y) :
    path_in_C (c + x) (c + y) where
  toFun := fun t => c + γ.toFun t
  continuous_toFun := Continuous.add continuous_const γ.continuous_toFun
  source' := by { change c + γ.toFun 0 = c + x; have h : γ.toFun 0 = x := (γ :
     Path x y).source'; rw [h] }
  target' := by { change c + γ.toFun 1 = c + y; have h : γ.toFun 1 = y := (γ :
     Path x y).target'; rw [h] }
```

$$\gamma_c(t) = c + \gamma(t)$$

- `pathDeriv` (γ : path_in_C x y, $t \in \mathbb{R}$): The tangent vector $\gamma'(t) \in \mathbb{C}$, extracted via choice from `HasDerivAt_R_to_C_eps` extended to $\mathbb{R}$.

```
noncomputable def pathDeriv {x y : ℂ} (γ : path_in_C x y) (t : ℝ) : ℂ :=
  deriv_R_to_C_eps γ.extend t

/-- The direction of a path γ at time t is the argument of its derivative. -/
```

$$\gamma'(t) = \frac{d\gamma}{dt}$$

- `pathDirection` (γ : path_in_C x y, $t \in \mathbb{R}$): The argument (angle) of the tangent vector, $\arg(\gamma'(t))$.

```
noncomputable def pathDirection {x y : ℂ} (γ : path_in_C x y) (t : ℝ) : ℝ :=
  arg (pathDeriv γ t)

/-- The angle between two paths γ₁ and γ₂ at times t₁ and t₂ is the difference
    of their directions. -/
```

$$\text{Direction}(\gamma, t) = \arg(\gamma'(t))$$

- `pathAngle` ($\gamma_1, t_1, \gamma_2, t_2$): The angle between two intersecting paths.

```
noncomputable def pathAngle {x₁ y₁ x₂ y₂ : ℂ} (γ₁ : path_in_C x₁ y₁) (t₁ : ℝ) (
    γ₂ : path_in_C x₂ y₂) (t₂ : ℝ) : ℝ :=
  pathDirection γ₂ t₂ - pathDirection γ₁ t₁

/--
  The angle between two paths is equal to the argument of the product of the
    second path's derivative
  and the conjugate of the first path's derivative, modulo 2π.
  We prove this equality here assuming the angle difference stays within the
    principal branch bounds.
-/
```

$$\text{Angle}(\gamma_1, t_1, \gamma_2, t_2) = \arg(\gamma_2'(t_2)) - \arg(\gamma_1'(t_1))$$

- `conformal` ($f : \mathbb{C} \to \mathbb{C}$, $z_0 \in \mathbb{C}$): f preserves angles between any two paths intersecting at z_0 with non-zero derivatives.

```
def conformal (f : ℂ → ℂ) (z₀ : ℂ) : Prop :=
  ∀ (x₁ y₁ x₂ y₂ : ℂ) (γ₁ : path_in_C x₁ y₁) (t₁ : ℝ) (γ₂ : path_in_C x₂ y₂) (t₂
    : ℝ) (d₁ d₂ : ℂ),
    γ₁.extend t₁ = z₀ →
    γ₂.extend t₂ = z₀ →
    HasDerivAt_R_to_C_eps γ₁.extend (pathDeriv γ₁ t₁) t₁ →
    HasDerivAt_R_to_C_eps γ₂.extend (pathDeriv γ₂ t₂) t₂ →
    pathDeriv γ₁ t₁ ≠ 0 →
    pathDeriv γ₂ t₂ ≠ 0 →
    -- HasDerivAt_R_to_C_eps f f' t says f' is the derivative of f at t.
    HasDerivAt_R_to_C_eps (f ∘ γ₁.extend) d₁ t₁ →
    HasDerivAt_R_to_C_eps (f ∘ γ₂.extend) d₂ t₂ →
    d₁ ≠ 0 ∧ d₂ ≠ 0 ∧ arg d₂ - arg d₁ = pathAngle γ₁ t₁ γ₂ t₂

/--
  \SII.11 Angle preservation at a point.  CONFORMALITY.
  If f is holomorphic at z₀ with non-zero derivative, then it preserves angles
     between regular paths intersecting at z₀.
-/
```

Conformal f at $z_0 \iff f$ preserves angles between all regular intersecting curves at z_0

- `line_path_zero` ($v \in \mathbb{C}$): A straight-line path from $-v$ to v passing through 0 at $t = 1/2$.

```
noncomputable def line_path_zero (v : ℂ) : path_in_C (-v) v where
  toFun := fun t => -v + (2 * ((t : ℝ) : ℂ)) * v
```

$$\gamma(t) = -v + 2tv = (2t - 1)v$$

- `centered_line_path` ($z, v \in \mathbb{C}$): A straight-line path from $z - v$ to $z + v$ evaluating to z at $t = 1/2$.

```
noncomputable def centered_line_path (z v : ℂ) : path_in_C (z - v) (z + v)
    where
  toFun := fun t => z + (2 * (t : ℝ) - 1) * v
```

$$\gamma(t) = z + (2t - 1)v$$

5 Sarason/Chapter3.lean

- `linFracTrans` ($a, b, c, d, z \in \mathbb{C}$): A linear-fractional transformation (Möbius transformation) on $\mathbb{C}$.

```
noncomputable def linFracTrans (a b c d z : ℂ) : ℂ :=
  (a * z + b) / (c * z + d)
```

$$\varphi(z) = \frac{az + b}{cz + d}$$

- φ ($a, b, c, d, z \in \mathbb{C}$): Sarason's shorthand notation for `linFracTrans`.

```
noncomputable abbrev φ (a b c d z : ℂ) : ℂ :=
  linFracTrans a b c d z
```

$$\varphi(a, b, c, d, z) = \frac{az + b}{cz + d}$$

- `linFracTransExt` ($a, b, c, d \in \mathbb{C}$, $w \in \mathbb{C} \cup \{\infty\}$): The extended linear-fractional transformation on the Riemann sphere $\mathbb{C} \cup \{\infty\}$ (`OnePoint` $\mathbb{C}$).

```
noncomputable def linFracTransExt (a b c d : ℂ) : OnePoint ℂ → OnePoint ℂ
  | (z : ℂ) => if c * z + d = 0 then ∞ else (linFracTrans a b c d z : ℂ)
  | ∞ => if c = 0 then ∞ else (a / c : ℂ)
```

$$\varphi(w) = \begin{cases} \frac{az+b}{cz+d} & \text{if } w = z \in \mathbb{C} \text{ and } cz + d \neq 0, \\ \infty & \text{if } w = z \in \mathbb{C} \text{ and } cz + d = 0, \\ \infty & \text{if } w = \infty \text{ and } c = 0, \\ \frac{a}{c} & \text{if } w = \infty \text{ and } c \neq 0. \end{cases}$$

- `IsLinFracTrans` ($\varphi : \mathbb{C} \cup \{\infty\} \to \mathbb{C} \cup \{\infty\}$): Predicate stating that φ is a non-degenerate linear-fractional transformation on $\mathbb{C} \cup \{\infty\}$.

```
def IsLinFracTrans (φ : OnePoint ℂ → OnePoint ℂ) : Prop :=
  ∃ a b c d : ℂ, a * d - b * c ≠ 0 ∧ φ = linFracTransExt a b c d
```

$$\text{IsLinFracTrans}(\varphi) \iff \exists a, b, c, d \in \mathbb{C},\ ad - bc \neq 0 \ \wedge\ \varphi(w) = \frac{aw + b}{cw + d}$$

- `gl2OfCoeffs` ($a, b, c, d \in \mathbb{C}$, $ad - bc \neq 0$): The invertible 2×2 matrix in $\mathrm{GL}(2, \mathbb{C})$ associated with coefficients (a, b, c, d).

```
noncomputable def gl2OfCoeffs (a b c d : ℂ) (h_det : a * d - b * c ≠ 0) : GL
    (Fin 2) ℂ :=
  Matrix.GeneralLinearGroup.mkOfDetNeZero !![a, b; c, d] ...
```

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \in \mathrm{GL}(2, \mathbb{C}), \quad ad - bc \neq 0$$

- `translationMatrix` ($b \in \mathbb{C}$):

```
noncomputable def translationMatrix (b : ℂ) : GL (Fin 2) ℂ :=
  Matrix.GeneralLinearGroup.mkOfDetNeZero !![1, b; 0, 1] ...
```

$$\begin{pmatrix} 1 & b \\ 0 & 1 \end{pmatrix}$$

- `scalingMatrix` ($a \in \mathbb{C}, a \neq 0$):

```
noncomputable def scalingMatrix (a : ℂ) (ha : a ≠ 0) : GL (Fin 2) ℂ :=
  Matrix.GeneralLinearGroup.mkOfDetNeZero !![a, 0; 0, 1] ...
```

$$\begin{pmatrix} a & 0 \\ 0 & 1 \end{pmatrix}$$

- `inversionMatrix`:

```
noncomputable def inversionMatrix : GL (Fin 2) ℂ :=
  Matrix.GeneralLinearGroup.mkOfDetNeZero !![0, 1; 1, 0] ...
```

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$